SERP Scraping Pipeline

Fetching Live Google Dance Shorts via SearchAPI into Azure Data Factory

A step-by-step build log with screenshots

Table of Contents

1. Project Overview

This project demonstrates a real-world SERP (Search Engine Results Page) scraping pipeline built entirely on Azure Data Factory (ADF). Instead of writing custom scraping code, the pipeline calls a hosted SERP API — SearchAPI.io's Google Shorts engine — which does the actual scraping of Google's search results for short-form videos (Shorts) on a given query, in this case "dance". ADF is used purely as the orchestration and storage layer: it calls the REST API on a schedule, receives the live JSON search results, and lands them into an Azure Data Lake Storage Gen2 (ADLS Gen2) container as timestamped JSON files.

The end-to-end flow is:

- A REST Linked Service in ADF authenticates and connects to the SearchAPI.io endpoint.
- A REST Dataset defines the specific API call — the Google Shorts engine, the search query ("dance"), and other parameters — as the Relative URL.
- An Azure Data Lake Storage Gen2 Linked Service and a JSON Dataset define where the results should be written.
- A Copy Data activity inside a Pipeline reads from the REST dataset (source) and writes to the ADLS dataset (sink).
- Dynamic expressions generate a unique, timestamped file name and folder path for every pipeline run, so each run's results are captured without overwriting previous ones.

The sections below walk through the build in the order it was actually implemented, with a screenshot and explanation for every major step.

2. Understanding the Data Source: SearchAPI.io — Google Shorts Engine

Before building anything in Azure, it's important to understand the data source. SearchAPI.io is a third-party service that exposes search engine results (Google, Bing, YouTube, etc.) as a simple REST/JSON API — this is what "SERP scraping as a service" means. Rather than maintaining scrapers, headless browsers, or proxies, a single authenticated GET request returns structured, ready-to-use JSON.

For this project, the relevant endpoint is the Google Shorts engine:

```
GET /api/v1/search?engine=google_shorts
```

This engine retrieves short-video results (from sources such as YouTube, Instagram, TikTok and Facebook) for a search query, and returns metadata like the title, source platform, author, video length, thumbnail, and link for each result. The required parameter is q — the search query — and it supports optional filters (time period, captions, etc.). The screenshot below shows the official documentation page for this engine.

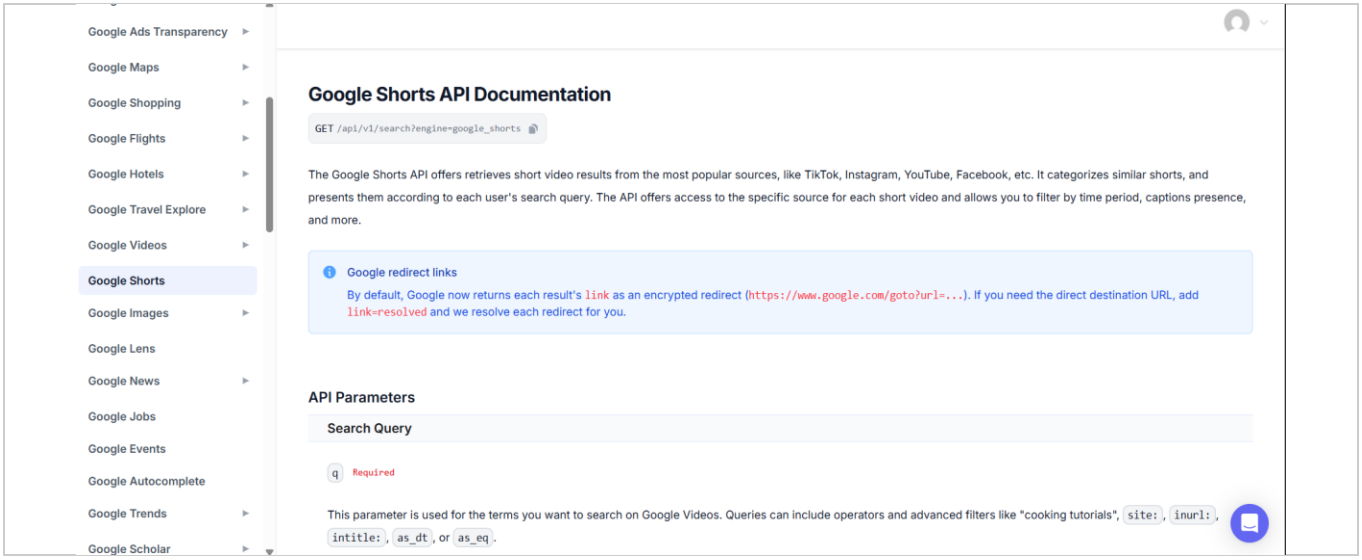

Fig 2.1 — SearchAPI.io documentation for the Google Shorts engine, showing the GET endpoint and the required 'q' (search query) parameter.

For this pipeline the query used is q=dance, so every run fetches the current, live "Google Dance Shorts" results — i.e. whatever short-form dance videos Google is currently surfacing for that search term.

3. Provisioning the Landing Zone: Azure Storage Account

The first infrastructure step is creating an Azure Storage Account that will act as the data lake landing zone for the scraped JSON results. From the Azure Marketplace, a new Storage Account resource is created under the desired subscription and resource group.

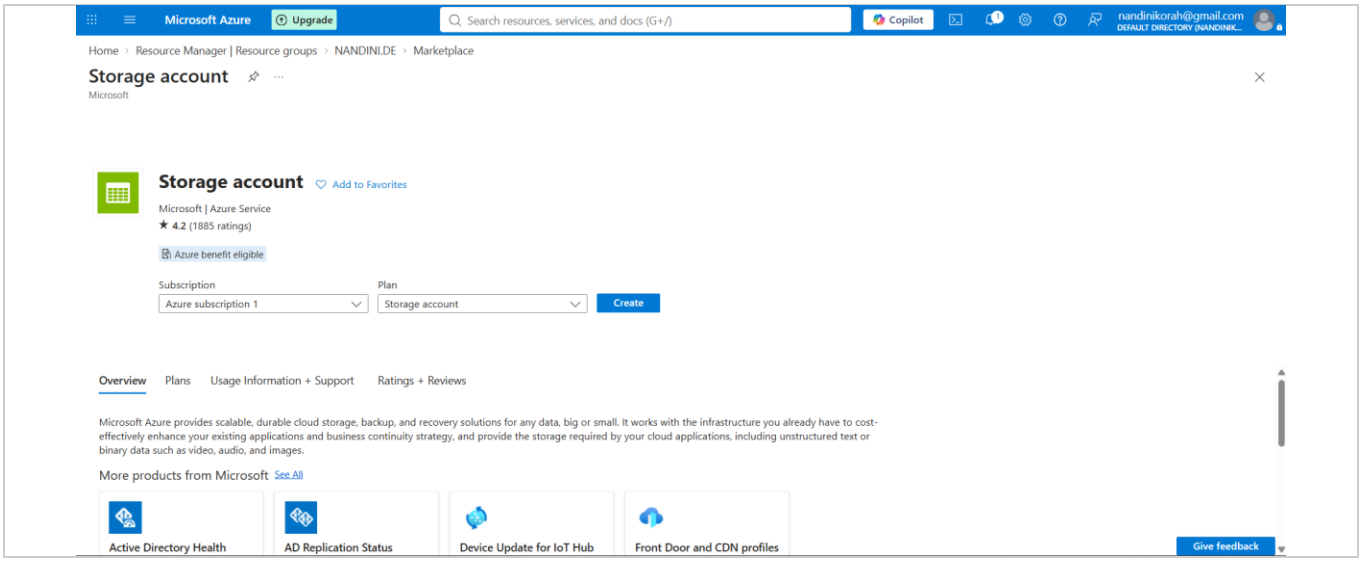

Fig 3.1 — Creating a new Storage Account resource from the Azure Marketplace.

The account (named storenans66 in this build) is provisioned as a general-purpose v2 StorageV2 account with Hierarchical Namespace enabled — this is what turns a standard Blob Storage account into an Azure Data Lake

Storage Gen2 account, which ADF's ADLS Gen2 connector expects. Key configuration to note in the Overview blade:

- Performance: Standard, Replication: Locally-redundant storage (LRS) — sufficient for a demo/dev pipeline.
- Data Lake Storage → Hierarchical namespace: Enabled.
- Security → Storage account key access: Enabled (used for the linked service's Account key authentication).

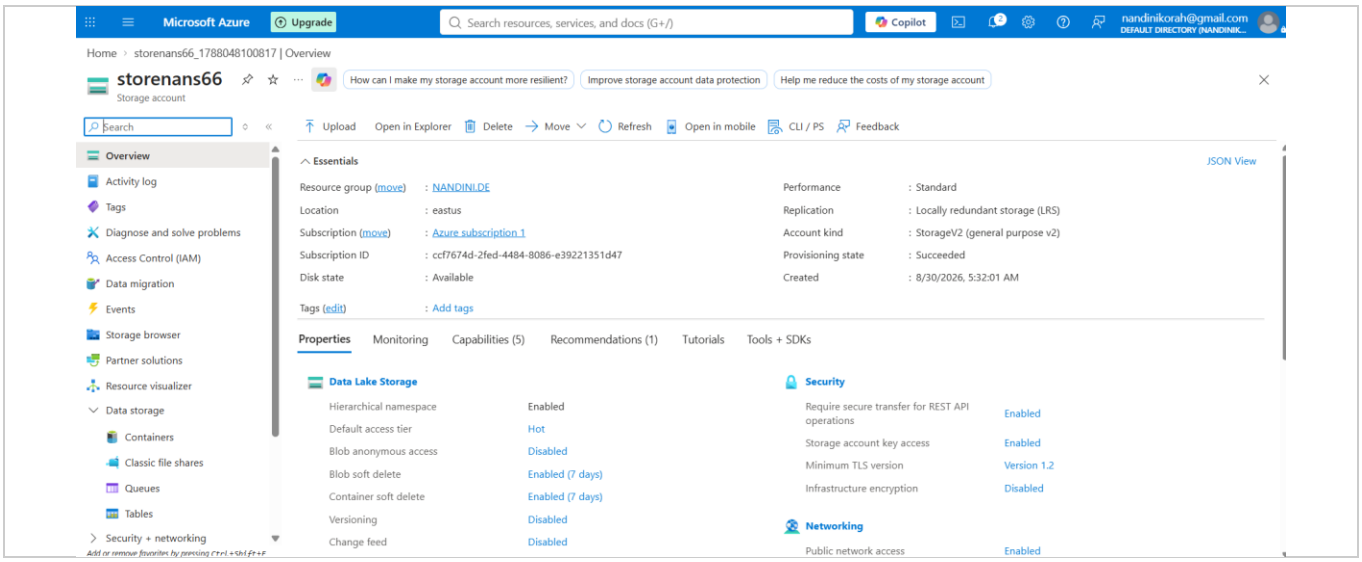

Fig 3.2 — Storage account overview confirming Hierarchical namespace is enabled and the account is provisioned successfully.

4. Creating the Blob Container

Inside the storage account, a dedicated container named output is created to hold the pipeline's results, keeping them separate from the account's default $logs container.

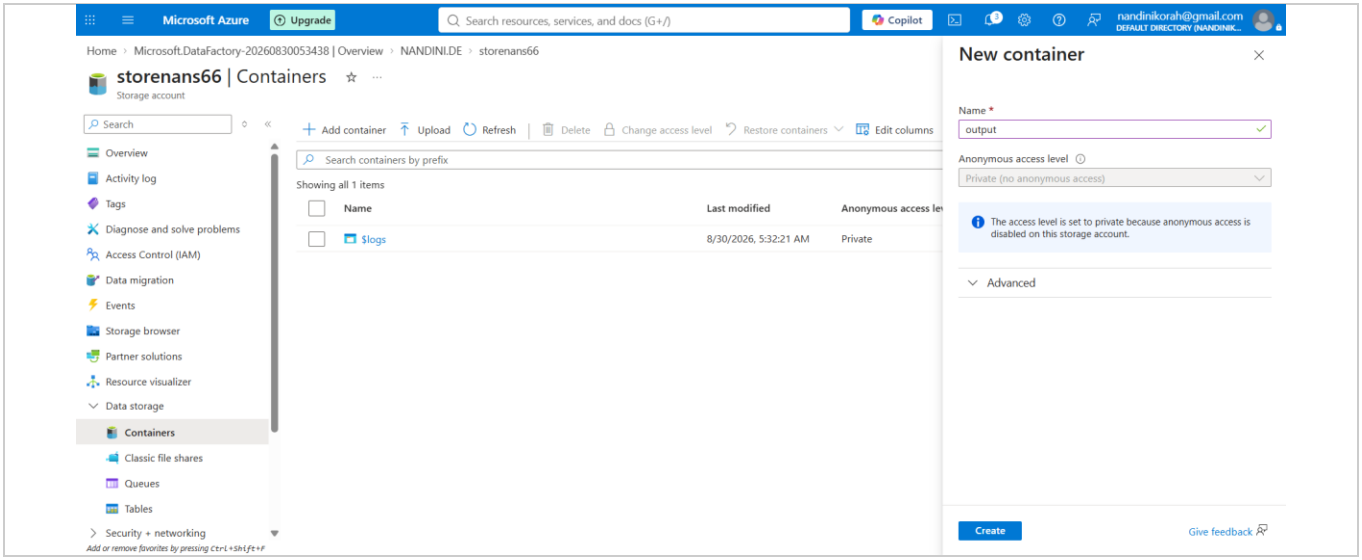

Fig 4.1 — Creating a new container named 'output' with a Private (no anonymous access) access level.

Once created, the Containers blade lists both the auto-generated $logs container and the newly created output container, confirming the landing zone is ready to receive files.

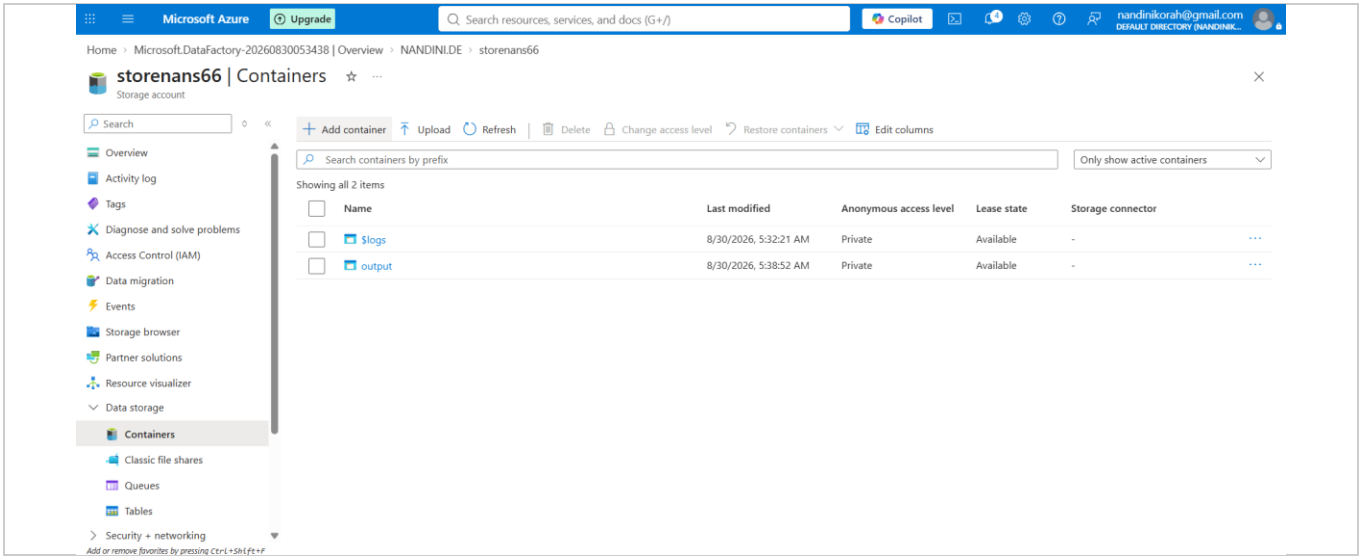

Fig 4.2 — Containers list showing the 'output' container alongside the default '$logs' container.

5. Provisioning Azure Data Factory

With storage in place, an Azure Data Factory instance (datafactnan66) is created. ADF is the orchestration engine of this project — it will define the connections, the datasets, and the pipeline that performs the actual copy operation from the API to storage.

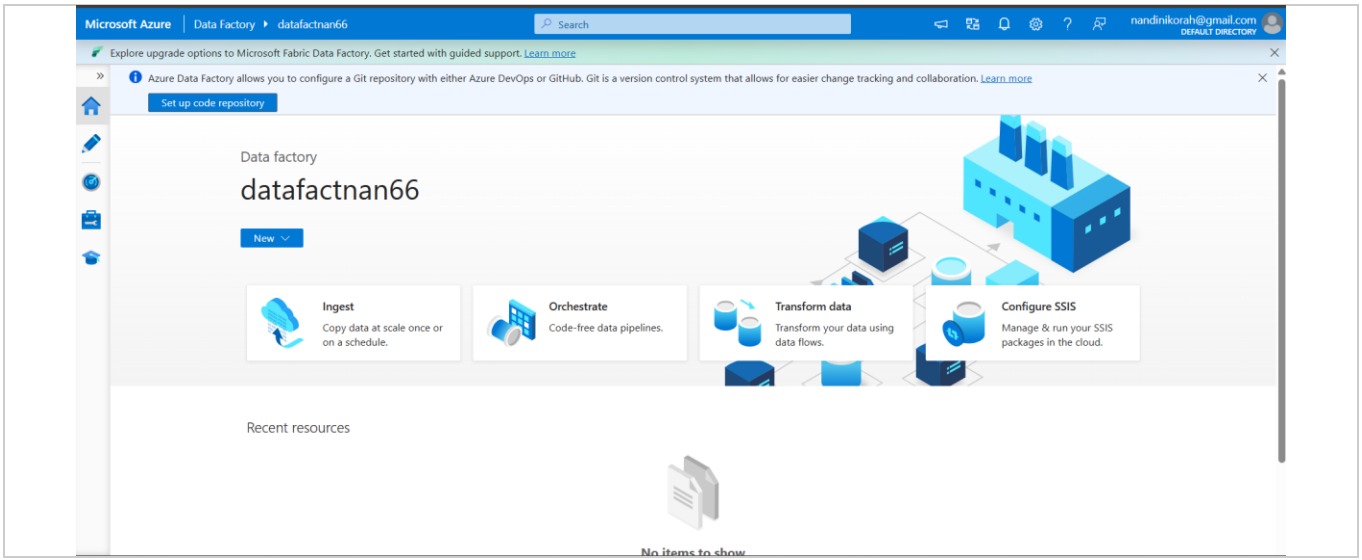

Fig 5.1 — The newly provisioned Data Factory landing page, ready for authoring pipelines.

From this Overview page, the Author (pencil) icon on the left-hand rail is used to open the ADF authoring canvas, where Linked Services, Datasets, and Pipelines are all created.

6. Creating the REST Linked Service (Source)

A Linked Service is how ADF stores a reusable connection definition. The first linked service, linkedservicerest1, is of type REST and points to the SearchAPI.io base URL.

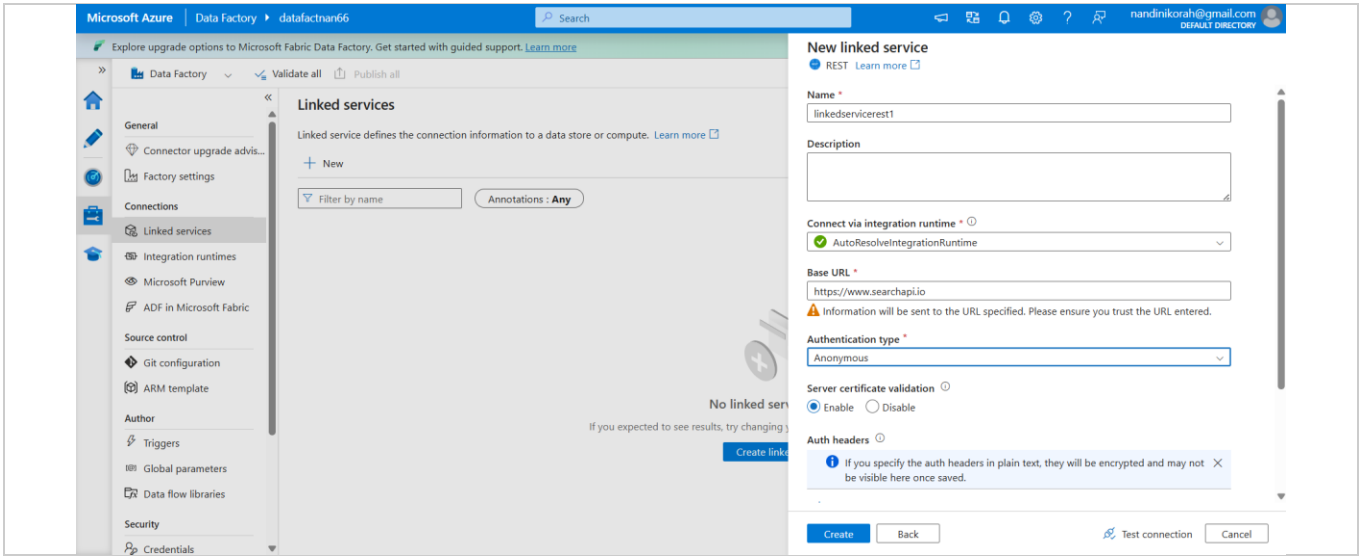

Fig 6.1 — New REST Linked Service configuration: Base URL set to https://www.searchapi.io, Authentication type 'Anonymous' (the API key is instead passed as a query parameter at the dataset level).

Key fields configured here:

- Base URL: https://www.searchapi.io — the root of every API call; the specific engine/query is appended later at the dataset level.
- Authentication type: Anonymous — no headers are required for this API's authentication style; the API key travels as part of the query string instead.
- Connect via integration runtime: AutoResolveIntegrationRuntime — ADF's managed, serverless compute for the copy operation.

7. Creating the ADLS Gen2 Linked Service (Sink)

A second linked service, linkedserviceadf2, connects ADF to the Azure Data Lake Storage Gen2 account created in Section 3. Authentication uses Account key, resolved automatically by selecting the subscription and storage account name (storenans66) from the dropdowns — Azure retrieves the key on ADF's behalf so it never needs to be typed manually.

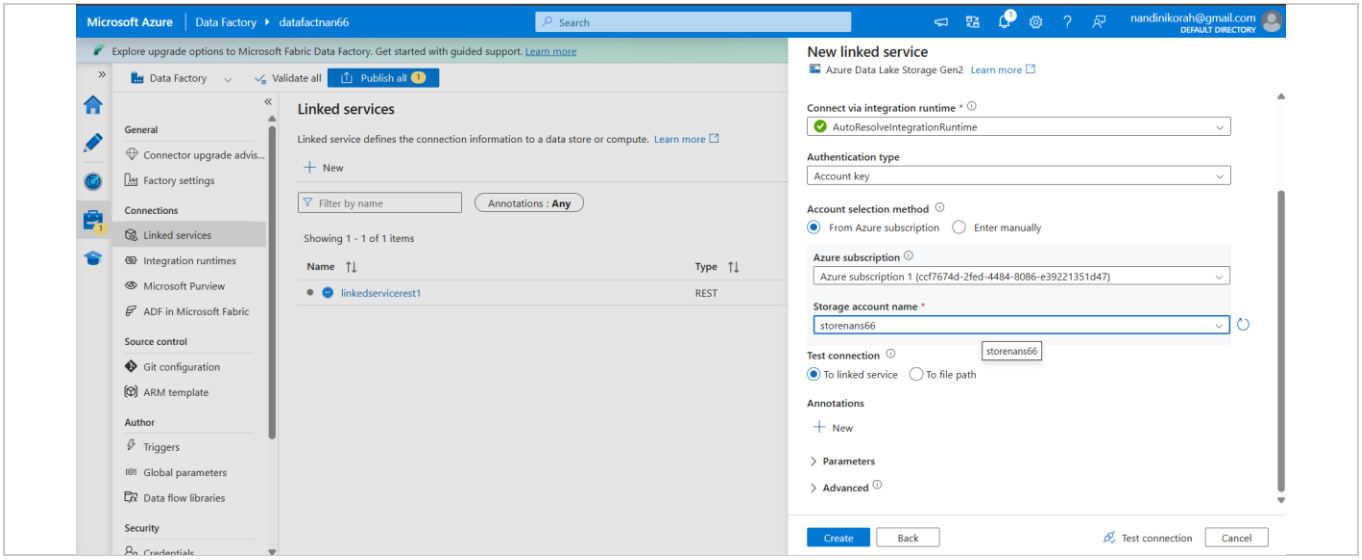

Fig 7.1 — New Azure Data Lake Storage Gen2 Linked Service, connected via Account key to the storenans66 storage account.

After both linked services are created, they appear together in the Linked Services list — one REST connection (the source/API) and one ADLS Gen2 connection (the sink/storage):

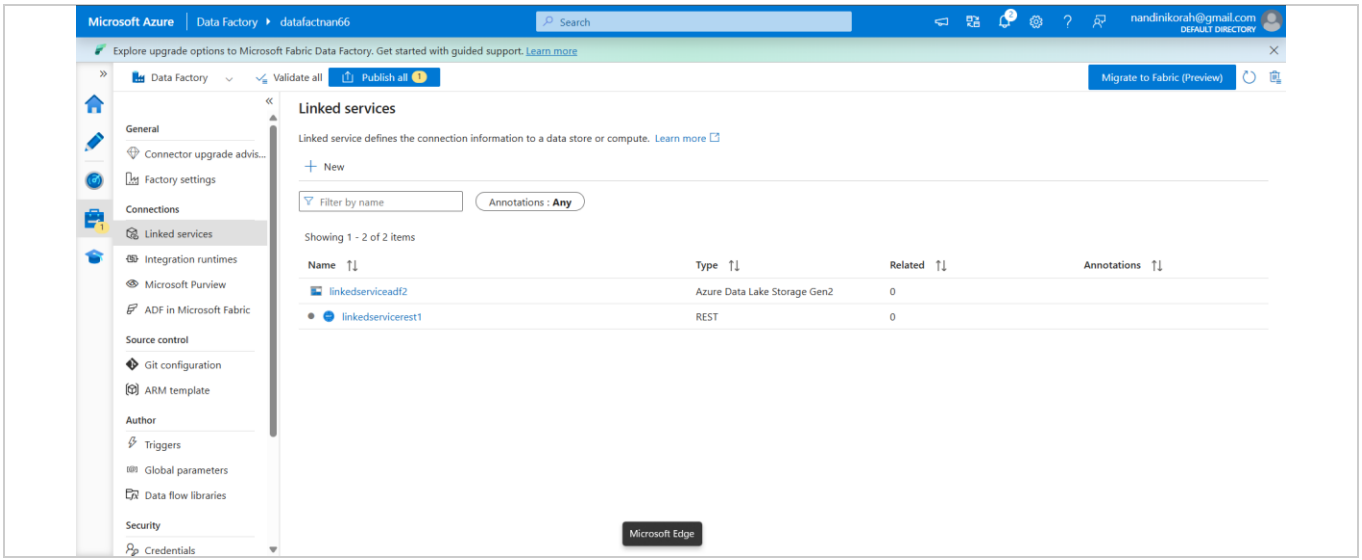

Fig 7.2 — Linked Services list showing 'linkedserviceadf2' (Azure Data Lake Storage Gen2) and 'linkedservicerest1' (REST), the two connections that anchor the whole pipeline.

8. Building the REST Source Dataset

A Dataset in ADF describes a specific view of data inside a Linked Service's connection — in this case, the exact API call to make. The dataset datasetrest1 uses linkedservicerest1 and defines a Relative URL that is appended to the Base URL to form the full SearchAPI.io request.

8.1 Composing the dynamic Relative URL

Because the request should include a timestamp component (so repeated runs are traceable) and the fixed engine/query parameters, the Relative URL is built using ADF's Pipeline/Dataset Expression Builder rather than typed as a flat string:

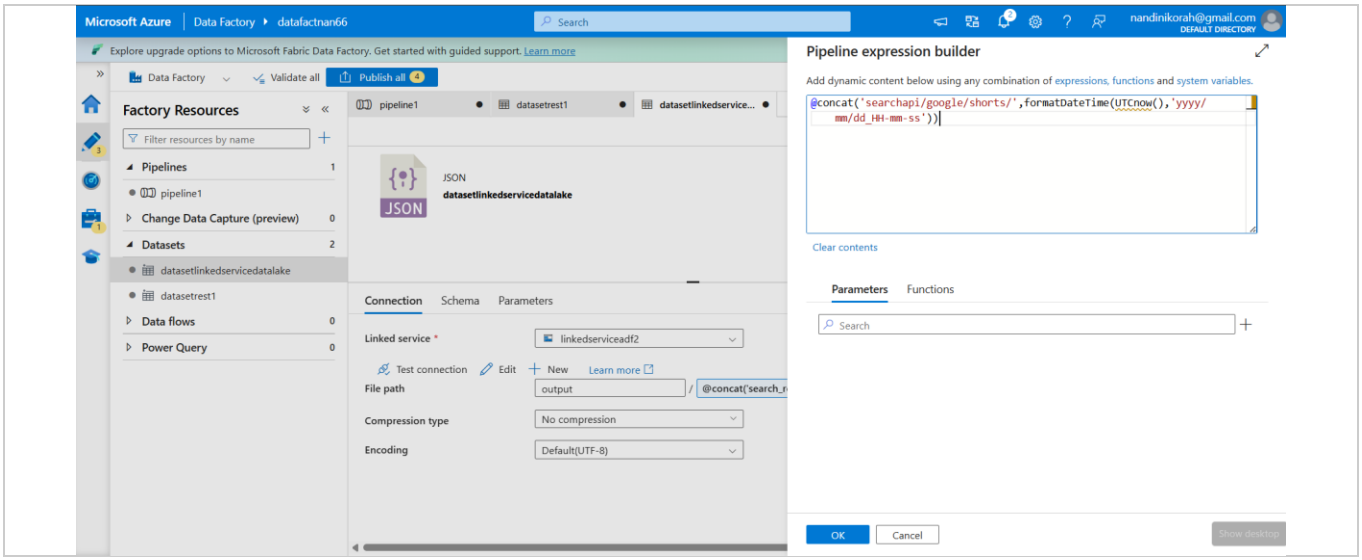

Fig 8.1 — Expression Builder composing the Relative URL: @concat('searchapi/google/shorts/', formatDateTime(UTCnow(),'yyyy/MM/dd_HH-mm-ss')).

This expression dynamically inserts the current UTC date and time (formatted as yyyy/MM/dd_HH-mm-ss) into the path, which is then combined elsewhere with the actual query-string parameters (engine=google_shorts, q=dance, and the SearchAPI api_key) to form the final request URL.

8.2 Verifying the connection

With the Relative URL configured, Test Connection confirms ADF can successfully reach the endpoint, and the dataset's Connection tab shows the resolved Base URL and Relative URL used for the call:

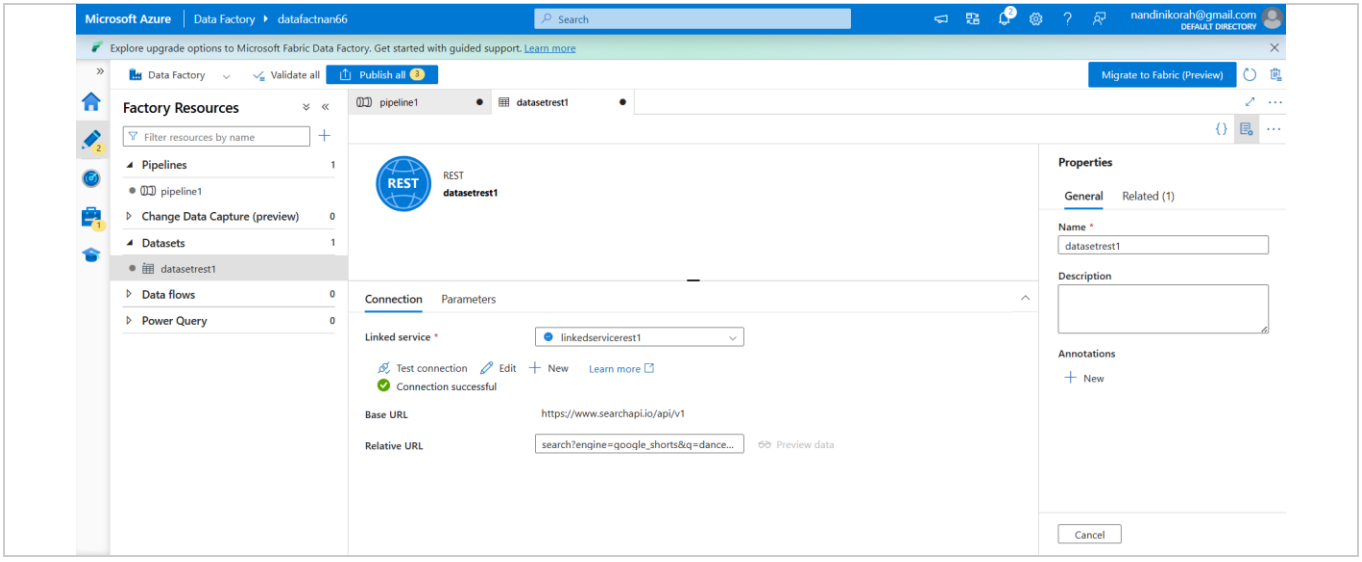

Fig 8.2 — datasetrest1 connection tab: Base URL https://www.searchapi.io/api/v1, Relative URL search?engine=google_shorts&q=dance..., with 'Connection successful' confirmed.

9. Previewing the Live API Response

Before wiring the dataset into a pipeline, the Preview data feature is used to confirm the API returns the expected structure. This makes a live call to SearchAPI.io through the REST linked service and displays the raw JSON response inside ADF.

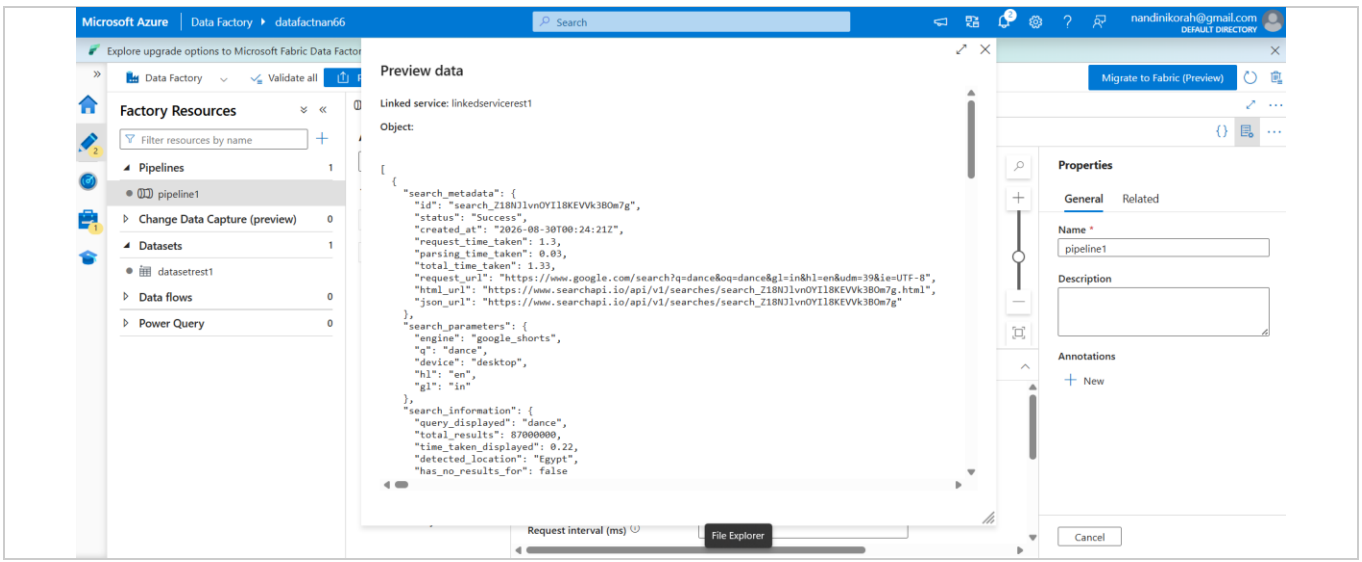

Fig 9.1 — Preview data window showing the live JSON response, including search_metadata, search_parameters (engine: google_shorts, q: dance), and the results array.

This confirms the response includes the top-level objects expected from the Google Shorts engine: search_metadata (request status and timings), search_parameters (the query actually sent), search_information (result counts), and the shorts array itself, which is the payload of interest.

10. Building the ADLS Gen2 Sink Dataset (JSON)

A second dataset, datasetlinkedservicedatalake, is created on top of linkedserviceadf2. Its type is JSON, and its File path targets the output container. Just as with the source URL, the file name is generated dynamically using an expression so every pipeline run produces a distinct, timestamped file instead of overwriting the previous result.

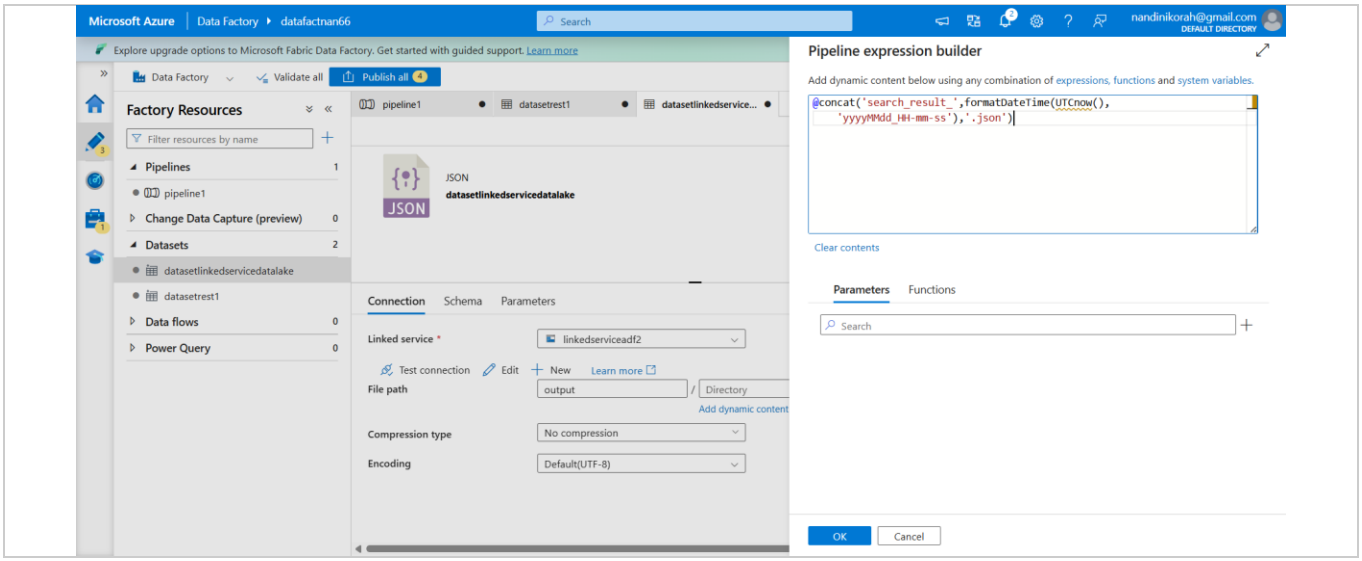

Fig 10.1 — Expression Builder composing the output file name: @concat('search_result_', formatDateTime(UTCnow(),'yyyyMMdd_HH-mm-ss'), '.json').

The resulting file path is output/<folder-path>/search_result_<timestamp>.json — combining the folder structure from the source Relative URL logic with a uniquely named JSON file for each run, giving a fully auditable, timestamped history of every scrape.

11. Assembling the Pipeline: Copy Data Activity

With both datasets defined, pipeline1 is created containing a Copy data activity. The activity's Source is set to datasetrest1 (the live Google Shorts API call) and its Sink is set to datasetlinkedservicedatalake (the timestamped JSON file in ADLS Gen2). No transformation is applied — this is a direct, schema-agnostic copy of the raw API JSON response into the data lake.

The pipeline is executed in Debug mode to validate the end-to-end flow without publishing:

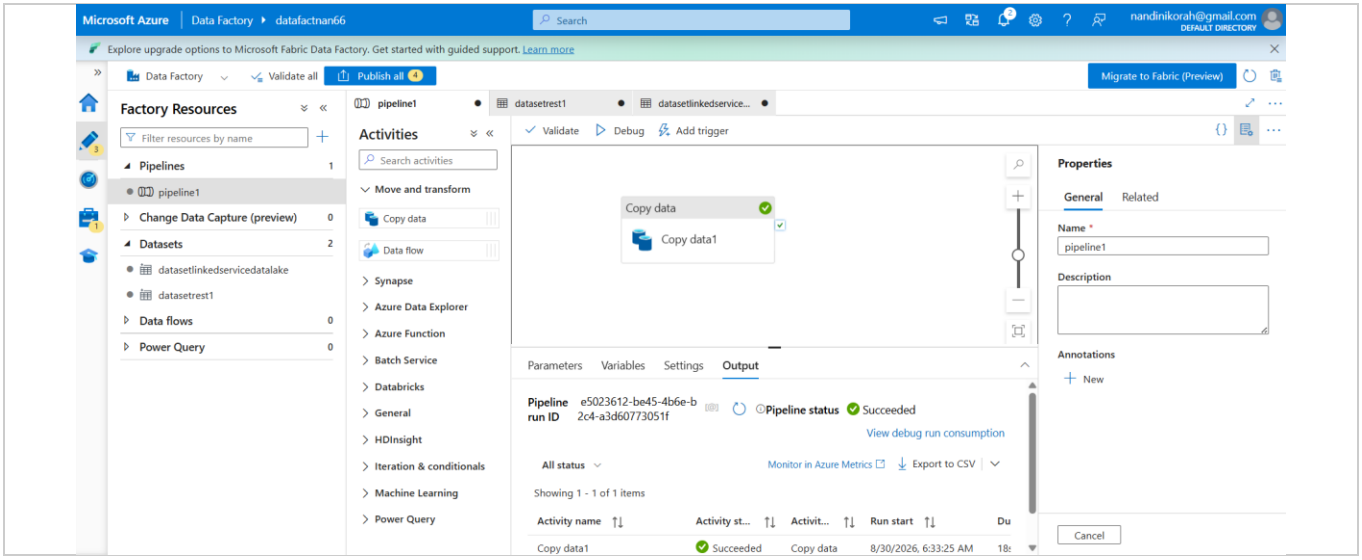

Fig 11.1 — Pipeline canvas with the Copy data (Copy data1) activity; the Output tab confirms the debug run finished with status 'Succeeded'.

The Output tab at the bottom of the canvas lists the pipeline run ID and each activity's individual status. Both the pipeline run and the Copy data1 activity report Succeeded, confirming the API call completed and the file was written to storage without errors.

12. Validating the Pipeline Output

After a successful run, the sink dataset's Preview data is used again — this time reading back the file that the pipeline just wrote — to confirm the copied JSON matches the live API response and that the shorts array (the actual scraped Google Dance Shorts results) is intact, including result position, title, video source (e.g. Instagram, YouTube), author, length, and thumbnail.

Fig 12.1 — Preview data of the written output file: search_information confirms query_displayed: "dance", and the shorts array lists individual results with position, title, source, author, length and thumbnail.

13. Verifying the File in Storage

Switching to the Storage Account's Containers blade, the output container now contains the folder structure generated by the dynamic Relative URL/file name expressions (searchapi/google/shorts/<date>/<time>/) and the resulting JSON file, confirming the pipeline correctly persisted a live, timestamped snapshot of the Google Shorts search results.

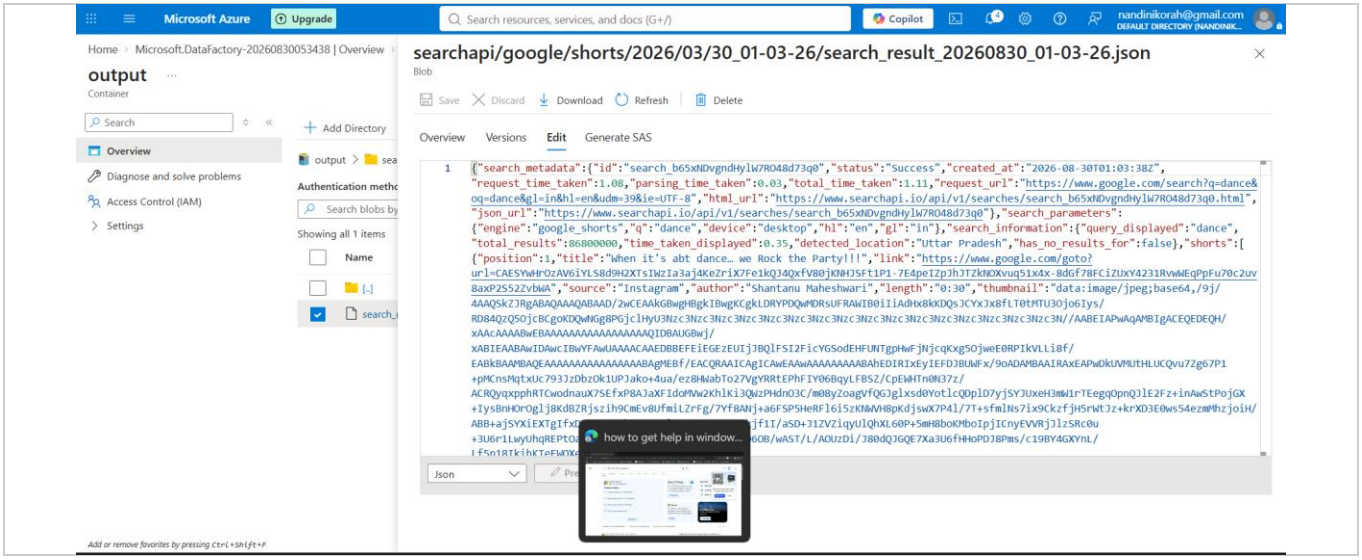

Fig 13.1 — Azure Storage Explorer 'Edit' view of the generated file (search_result_20260830_01-03-26.json) inside output/searchapi/google/shorts/2026/03/30_01-03-26/, showing the same JSON payload observed in the pipeline preview.

14. Sanity-Checking the JSON with an External Viewer

As a final validation step, the raw JSON content is pasted into a standalone JSON viewer/formatter tool. This confirms the file is well-formed JSON and makes it easy to visually inspect the nested structure — search_metadata, search_parameters, search_information, and the shorts array — outside of the Azure Portal.

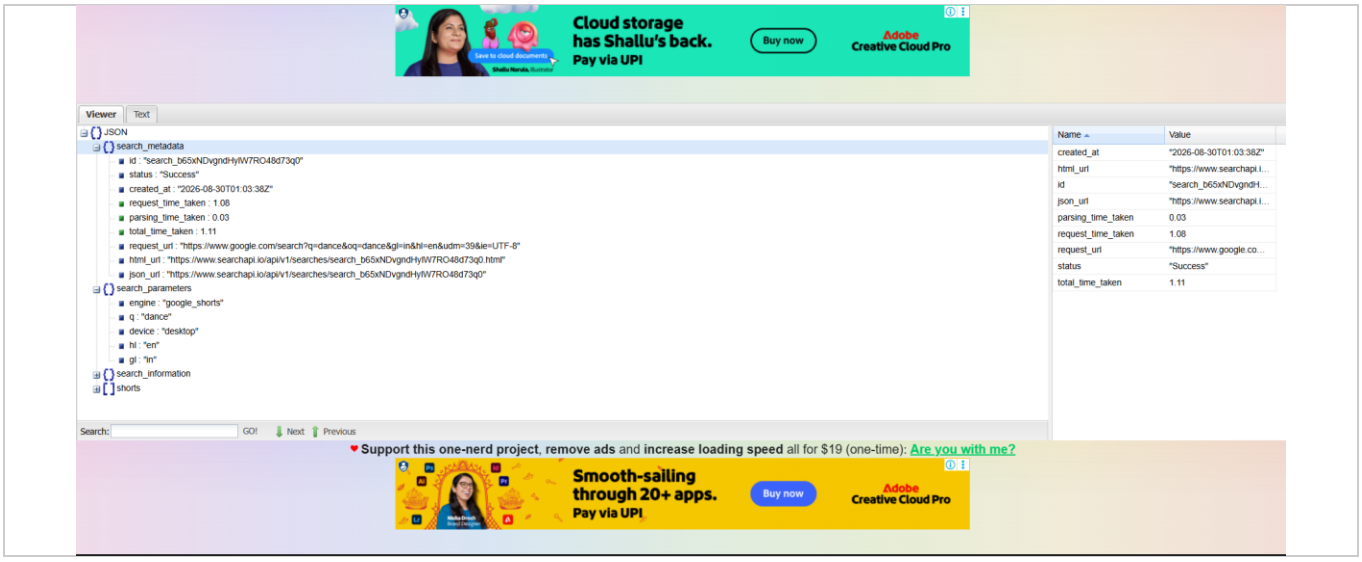

Fig 14.1 — External JSON viewer confirming the scraped file is valid JSON, with search_metadata (id, status: 'Success', timings) and search_parameters (engine: 'google_shorts', q: 'dance') clearly structured.

15. Summary

This build shows how Azure Data Factory can orchestrate a third-party SERP API into a repeatable, timestamped data-lake ingestion pipeline, with no custom scraping or scripting code required inside Azure:

- A REST Linked Service + Dataset call SearchAPI.io's Google Shorts engine live, using a dynamically built Relative URL.
- An ADLS Gen2 Linked Service + JSON Dataset write the raw response to a dedicated 'output' container, with a dynamically generated, timestamped file name.
- A single Copy data activity inside a pipeline connects the two, and Debug runs / Preview data confirm the flow end-to-end before publishing.
- Every run is preserved as its own JSON file, forming an auditable historical archive of Google's live Shorts results for the tracked query.

From here, the pipeline could be extended with a Tumbling Window or Schedule trigger to run on a fixed cadence, a Data Flow to parse/flatten the shorts array into tabular form, or a downstream copy into a database or Synapse for analytics on trending content.